Build 03 Flight Controller

A cascaded multicopter controller for ROS 2 and Gazebo, explained

Safwan Shaikh · `my_drone` 0.3.0

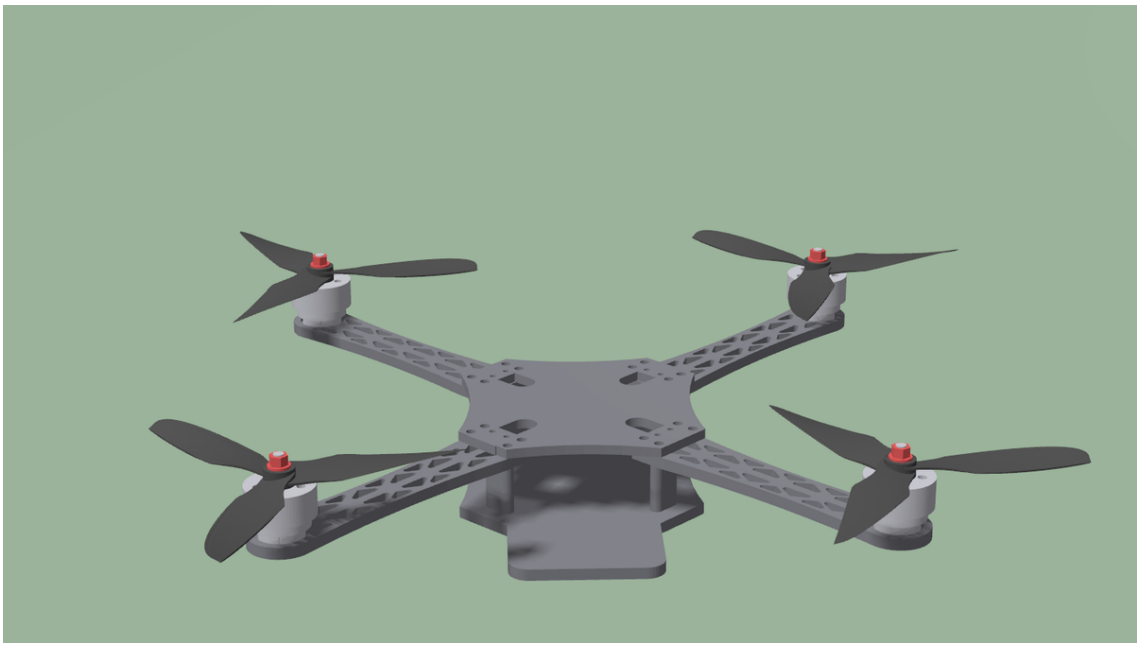


Figure 1. Build 03 hovering in Gazebo Harmonic under its own flight controller.

Summary. Build 03 is a 7-inch, 0.93 kg quadcopter: 2807 motors, 7040 tri-blade props, 6S. In simulation it used to fly on Gazebo's built-in velocity controller, a black box with no position loop. This document describes the controller that replaced it. It is the same cascade PX4 and ArduPilot use (position → velocity → thrust vector → attitude → body rate → mixer), written as one ROS 2 node in C++. Each loop is derived, tied to the code that implements it, and shown working on data recorded from the simulator.

1. Why a cascade
2. The vehicle model
3. Mixer
4. Rate loop
5. Attitude loop
6. Thrust vector and tilt limit
7. Velocity loop
8. Position loop
9. Modes, estimator and safety
10. Results
11. Compared with Gazebo's controller
12. Tuning guide

1 Why a cascade

A quadcopter has four inputs (the motor speeds) but six degrees of freedom. All four props push along one axis, the body z axis, so the vehicle cannot push sideways; it has to *tilt* its thrust. That forces a chain:

$$\textit{position} \rightarrow \textit{velocity} \rightarrow \textit{acceleration} \rightarrow \textit{tilt} \rightarrow \textit{body rate} \rightarrow \textit{torque} \rightarrow \textit{motor speeds}$$

Each arrow is one controller, and each treats the loop inside it as a perfect actuator. That only holds if every inner loop is clearly faster than the one wrapped around it. The table below shows the separation used here: roughly a factor of three per stage.

Loop	Gain (physical units)	Rough bandwidth
Motors (physics)	time constant 20 ms	50 rad/s
Rate	K_p = 18 (rad/s² per rad/s)	18 rad/s
Attitude	K_p = 6 (rad/s per rad)	6 rad/s
Velocity	K_p = 2 (m/s² per m/s)	2 rad/s
Position	K_p = 1 (m/s per m)	1 rad/s

Loop bandwidths, inner to outer.

Architecture

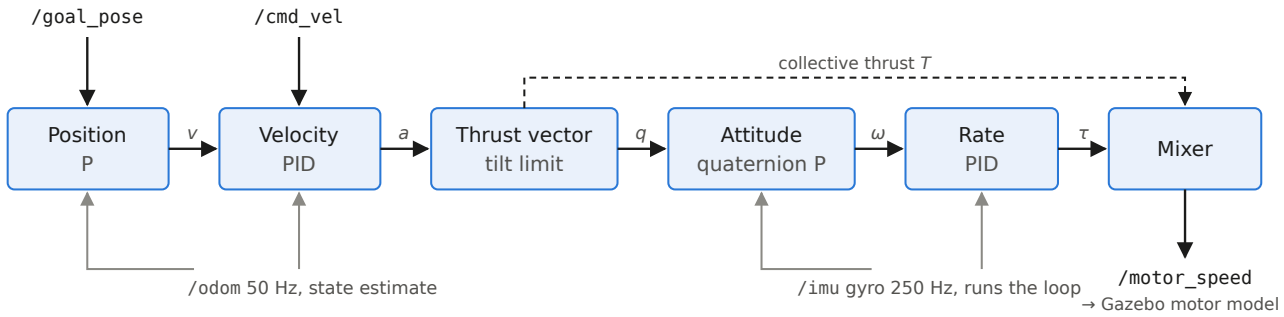


Figure 2. The cascade in `flight_controller`. A position setpoint enters on the left, a velocity setpoint (teleop) one block later. The collective thrust goes straight from the thrust-vector block to the mixer.

The code is split in two:

- `src/control.hpp`: the maths, free of ROS and unit tested (`test/test_control.cpp`).
- `src/flight_controller.cpp`: ROS input and output, setpoint modes, arming and failsafes.

Mass, inertia, motor constants and rotor layout are read from a `<quad_physics>` element that `quad_gazebo.xacro` writes into the URDF. The controller therefore always uses the same numbers as the simulated vehicle. Gains live in `config/controller.yaml`.

2 The vehicle model

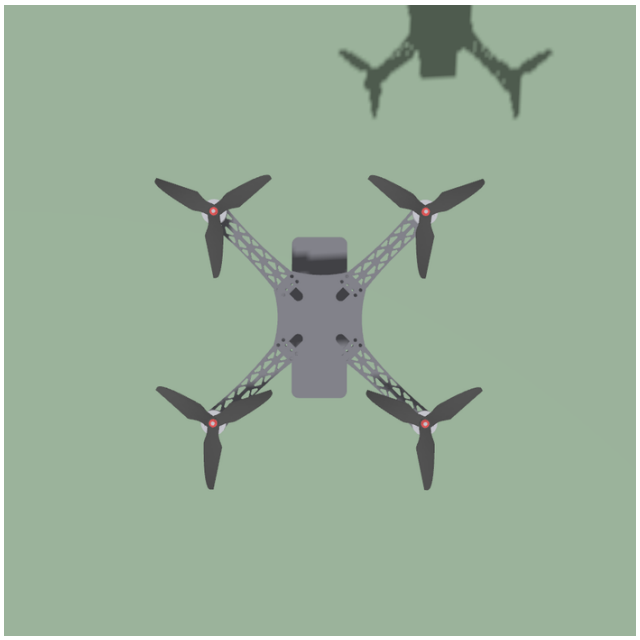
Gazebo's `MulticopterMotorModel` turns a commanded rotor speed ω into thrust and a drag torque on the body:

$$f = k\omega^2, \qquad \tau_{\text{drag}} = c\,f, \qquad k = 3.7\times10^{-6}\,\text{N s}^2/\text{rad}^2, \qquad c = 0.016\,\text{m}$$

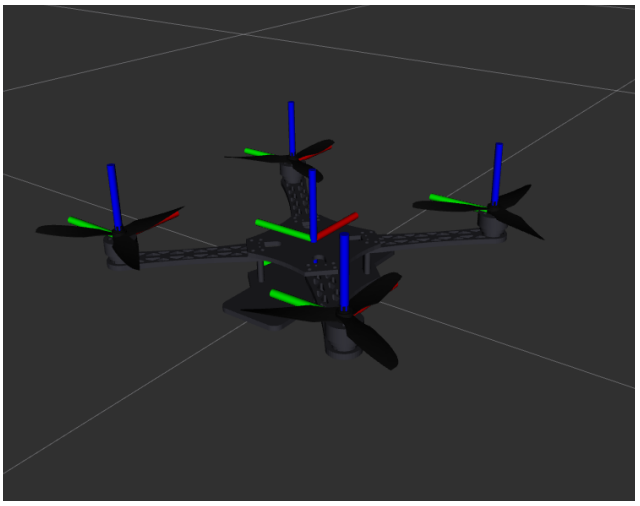
Hover needs $mg/4$ = 2.28 N per rotor, so $\omega_{\text{hover}} = \sqrt{(2.28/k)} \approx 785$ rad/s. That is exactly the speed the rotors settle at in the simulator. Full thrust is 19.6 N per rotor, which gives a thrust-to-weight ratio of 8.6. The vehicle is heavily overpowered, which is why the position loops are capped (Section 6).

Rotation follows Euler's rigid-body equation, with $\boldsymbol{J}$ the inertia (I_{xx} = 0.00711, I_{yy} = 0.00754, I_{zz} = 0.01362 kg m²):

$$\boldsymbol{\tau} = \boldsymbol{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\boldsymbol{J}\boldsymbol{\omega})$$



Top view. Rotor 0 front-right (ccw), 1 rear-left (ccw), 2 front-left (cw), 3 rear-right (cw).



RViz: body frame at the centre of mass and one frame per rotor (x red, y green, z blue).

Figure 3. Build 03's layout: a true X, rotors 143 mm from the centre of mass on each axis.

3 Mixer: from thrust and torques to motors

The controller asks for a collective thrust T and body torques τ_x, τ_y, τ_z. Rotor i sits at (x_i, y_i) and spins with direction d_i (+1 counter-clockwise). Its thrust f_i contributes:

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ y_0 & y_1 & y_2 & y_3 \\ -x_0 & -x_1 & -x_2 & -x_3 \\ -d_0 c & -d_1 c & -d_2 c & -d_3 c \end{bmatrix}}_{A} \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ f_3 \end{bmatrix}$$

The rows read as follows:

- **Roll**: lift on the left (+y) rolls the right side down.
- **Pitch**: lift at the back pitches the nose down.
- **Yaw**: a prop spinning counter-clockwise drags the body clockwise, hence the minus sign.

A is built from the rotor geometry and inverted once at start-up, so $\boldsymbol{f} = A^{-1}[T, \boldsymbol{\tau}]^\top$ and $\omega_i = \sqrt{(f_i/k)}$. Building A from geometry means any quad layout works without a hand-written mixer table.

Yaw is the weak axis. Roll and pitch act through a 0.143 m lever; yaw only through c = 0.016 m,

nine times less. Near hover a large yaw demand asks for more thrust difference than the motors have. The first version of the mixer dropped yaw entirely when that happened, and the drone stalled 62° short of its yaw target.

The fix follows PX4. Thrust, roll and pitch are served first. Yaw then gets the largest fraction $s \in [0, 1]$ of its torque that keeps every rotor within its limits:

$$\boldsymbol{f} = A^{-1}\big[T, \tau_x, \tau_y, 0\big]^{\mathsf{T}} + s\, A^{-1}[0, 0, 0, \tau_z]^{\mathsf{T}}$$

Figure 4 shows this during a 90° turn. One diagonal pair is pushed down to the idle floor while the other speeds up, so total thrust stays constant. The pairs swap roles to brake the turn.

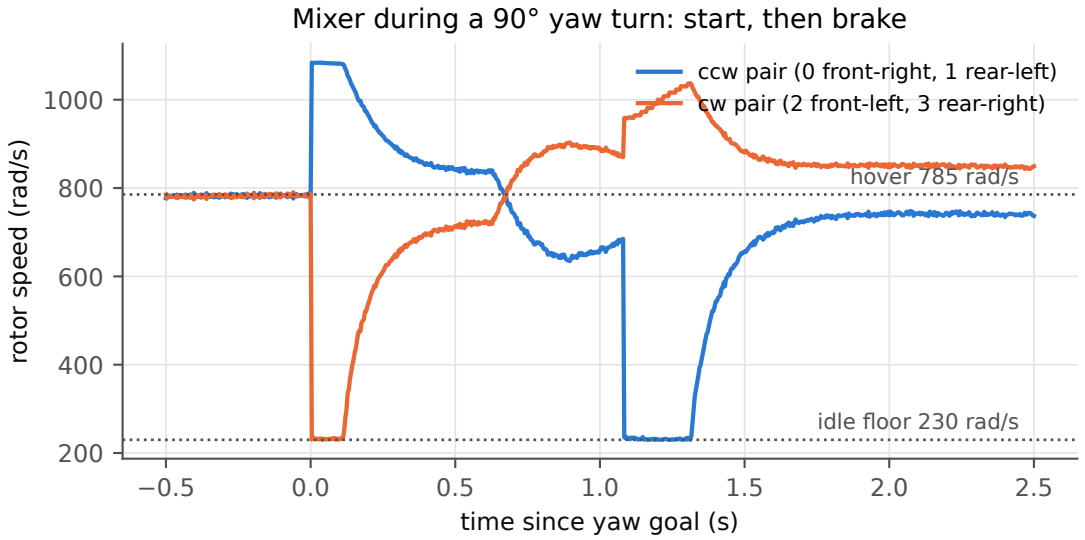


Figure 4. Rotor speeds during a 90° yaw. The turn saturates one pair at idle; $s < 1$ keeps the yaw command as large as still fits.

4 Rate loop

The innermost loop makes the body spin at the rate $\boldsymbol{\omega}_{\text{sp}}$ that the attitude loop asks for. It reads the gyro directly and runs on every IMU sample (250 Hz), the way a real flight controller is triggered.

$$\dot{\boldsymbol{\omega}}_{\text{sp}} = K_p(\boldsymbol{\omega}_{\text{sp}} - \boldsymbol{\omega}) + K_i \int (\boldsymbol{\omega}_{\text{sp}} - \boldsymbol{\omega})\, dt - K_d\, \dot{\boldsymbol{\omega}}, \qquad \boldsymbol{\tau} = \boldsymbol{J}\,\dot{\boldsymbol{\omega}}_{\text{sp}} + \boldsymbol{\omega} \times \boldsymbol{J}\,\boldsymbol{\omega}$$

What each part does:

- **P** reacts to the current rate error.
- **I** removes steady torque offsets: a centre of mass slightly off centre, or one motor slightly weaker. It is clamped (`rate_i_max`) so it cannot wind up while the drone sits on the ground.
- **D** acts on the *measured* angular acceleration, not on the error, so a step in the setpoint gives no derivative kick. It is low-pass filtered because differentiating a noisy gyro amplifies the noise.
- **Multiplying by $\boldsymbol{J}$** keeps the gains in physical units. A heavier or larger airframe changes J, and the torque scales with it; the gains do not need retuning.

5 Attitude loop

Orientation is kept as a quaternion. Euler angles have a singularity at 90° pitch and wrap at ±180°; a quaternion has neither problem. The error rotation, expressed in body axes, is

$$q_e = q^{-1} \otimes q_{\text{sp}}, \qquad \boldsymbol{\omega}_{\text{sp}} = 2\,K_p\,\text{sign}(q_{e,w})\,\boldsymbol{q}_{e,xyz}$$

The result is saturated at `rate_max`.

A quaternion's vector part is $\boldsymbol{n}\,\sin(\theta/2)$. For small errors $2\sin(\theta/2) \approx \theta$, so the law is simply "spin rate = gain × angle error", about the correct axis. The sign term picks the shorter way round.

A P controller is enough here. The rate loop below behaves like an integrator (rate becomes angle), so the attitude loop has no steady-state error. Yaw gets a lower gain (3 instead of 6) because it has the least authority.

Between 50 Hz odometry updates the attitude estimate is propagated with the gyro. This is the prediction step of any real estimator:

$$q \leftarrow q \otimes \exp(\frac{1}{2}\,\boldsymbol{\omega}\,\Delta t)$$

6 Thrust vector and tilt limit

This block connects the "where to go" loops to the "how to point" loops. The velocity loop asks for a world acceleration $\boldsymbol{a}_{\text{sp}}$, and the props must also cancel gravity:

$$\boldsymbol{f} = \boldsymbol{a}_{\text{sp}} + g\,\hat{\boldsymbol{z}}$$

Here $\boldsymbol{f}$ is the specific thrust (thrust per unit mass, in the world frame). Its direction is where body z must point; its size is how hard to push.

Before use it is limited in PX4's priority order:

1. **Vertical first.** f_z is clamped to $[0.1g,\ a_{\max}]$, where $a_{\max}$ is 80% of full thrust. The rest is held back for the attitude loop.
2. **Horizontal from what is left.** The horizontal part is limited by the tilt limit and by the remaining thrust:

$$|\boldsymbol{f}_{xy}| \le \min\left(f_z \tan 35°,\ \sqrt{a_{\max}^2 - f_z^2}\right)$$

The target orientation is then built from two requirements: body z along $\boldsymbol{f}$, and the nose at the commanded yaw ψ. Body y is $\hat{z}_{\text{b}} \times (\cos\psi, \sin\psi, 0)$, and body x completes the frame.

The collective thrust uses the body z the vehicle has *now*:

$$T = m\,\boldsymbol{f} \cdot \hat{\boldsymbol{z}}_{b,\text{now}}$$

While it is still rotating toward the target it applies only the useful component of thrust, so it does not balloon upward mid-manoeuvre.

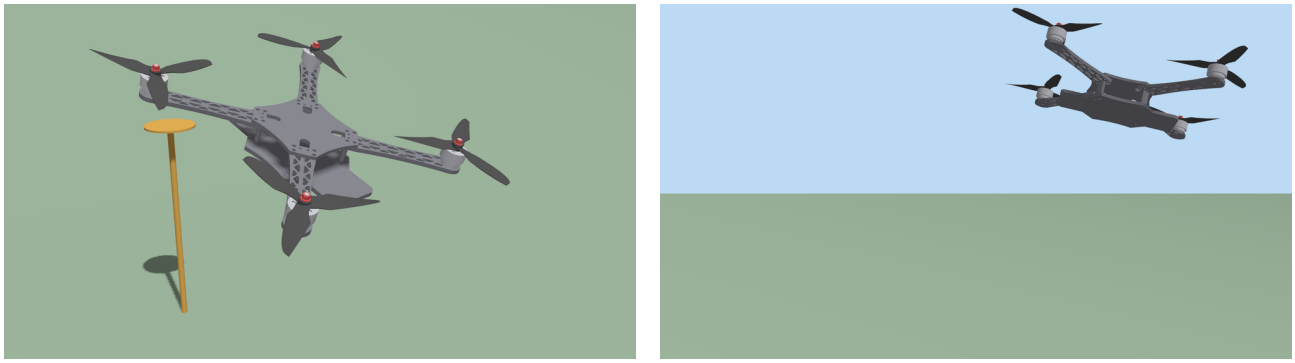


Figure 5. Accelerating between waypoints: the controller tilts the thrust vector to produce horizontal acceleration.

7 Velocity loop

$$\boldsymbol{a}_{\mathrm{sp}} = K_p(\boldsymbol{v}_{\mathrm{sp}} - \boldsymbol{v}) + K_i \int (\boldsymbol{v}_{\mathrm{sp}} - \boldsymbol{v})\, dt - K_d \boldsymbol{a}_{\mathrm{meas}}$$

- **P** sets the response: 1 m/s of velocity error asks for 2 m/s² (4 m/s² vertically).
- **I** absorbs whatever the model gets wrong: mass error (0.931 kg is an estimate until the parts are weighed), wind, and drag. Vertically the integrator effectively learns the true hover thrust, which is why it is stronger on z.
- **D** acts on the measured acceleration, differentiated from odometry, and only adds a little damping.

8 Position loop

$$\boldsymbol{v}_{\mathrm{sp}} = K_p(\boldsymbol{p}_{\mathrm{sp}} - \boldsymbol{p}), \qquad |\,\boldsymbol{v}_{\mathrm{sp},\,xy}\,| \le 3\ \mathrm{m/s}, \qquad -1 \le v_{\mathrm{sp},\,z} \le 2\ \mathrm{m/s}$$

A long move has two phases. While the error is large the speed cap is active, so the drone cruises at constant speed. Once the error falls below 3 m, the setpoint decays exponentially, by a factor of e per second with $K_\mathrm{p} = 1$.

There is no integral term. The velocity integrator already removes steady disturbances, and a second integrator would mostly add overshoot.

Figure 6 shows the whole cascade on one step:

1. the position error saturates the velocity setpoint;
2. the velocity loop asks for acceleration;
3. the thrust-vector block turns that into about 28° of pitch;
4. the drone pitches back to brake.

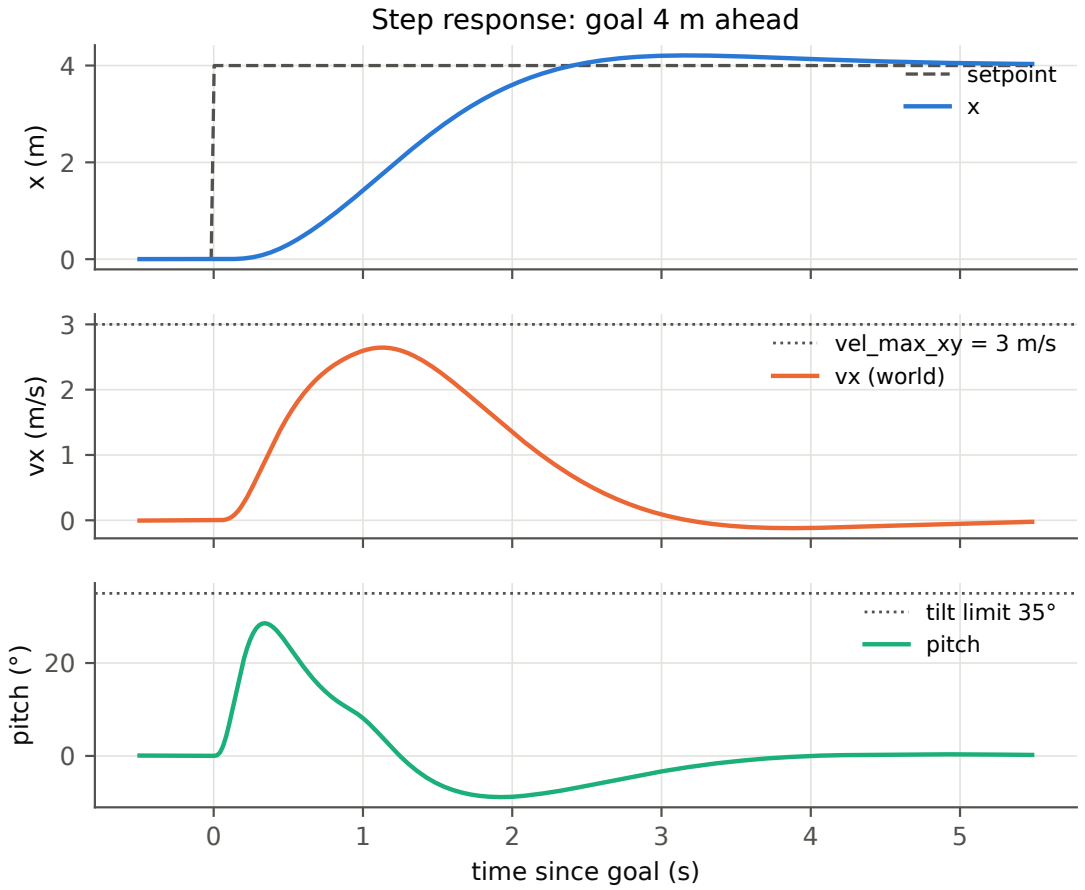


Figure 6. Step response, goal 4 m ahead. Arrives in 3.9 s with 0.21 m overshoot; pitch stays inside the 35° tilt limit.

9 Modes, estimator and safety

Setpoint modes. The input that arrived most recently wins, as in PX4's offboard mode:

- `/goal_pose` enters the cascade at the position loop.
- `/cmd_vel` (teleop) enters at the velocity loop.

When the stick is centred, or `/cmd_vel` goes quiet for 0.5 s, the drone first brakes with $\boldsymbol{v}_{\mathrm{sp}} = 0$. Once slower than 0.3 m/s it latches a position hold, so it stops and stays put instead of drifting. The yaw setpoint freezes the moment the yaw stick is released (Figure 7).

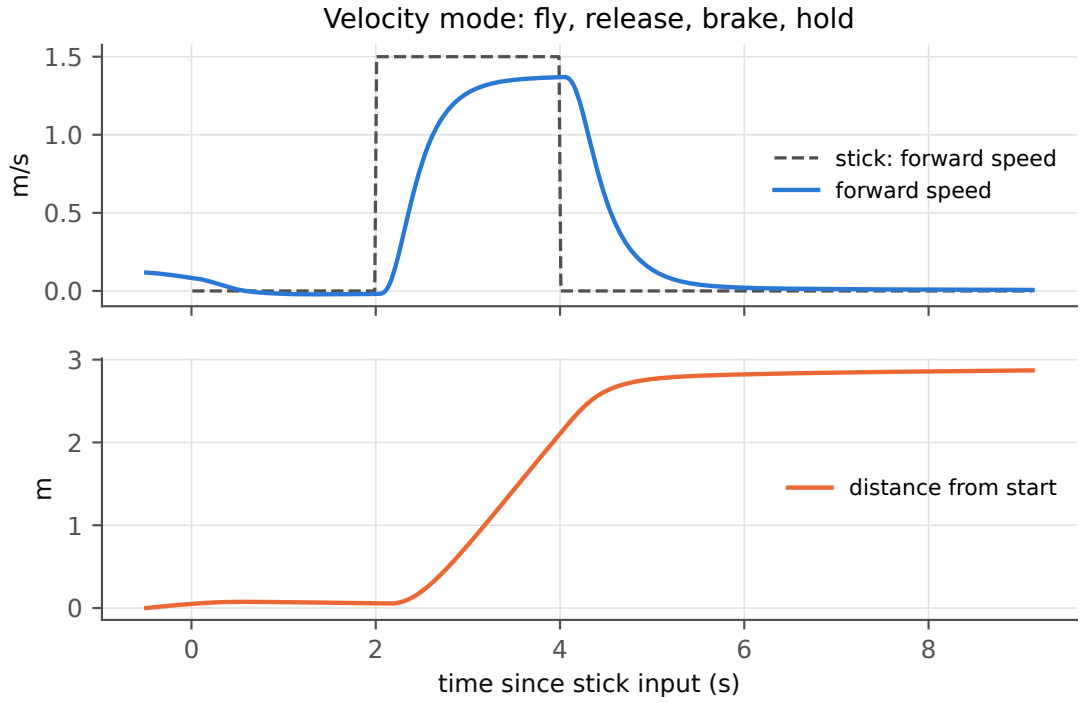


Figure 7. Velocity mode: 2 s of forward stick, then release. The drone brakes and holds position to a few centimetres.

Ground and arming. The controller does the following:

- Disarmed: all rotor speeds are zero.
- Armed on the ground: the props idle at 10% speed, and the integrators stay reset until something asks to climb, so nothing winds up before take-off.
- Disarming stops the motors immediately.

Failsafe. If odometry stops for more than 0.2 s, the drone holds level on the gyro-propagated attitude and sinks at 90% of hover thrust.

Estimator. `/odom` is currently Gazebo's ground truth. An EKF (for example `robot_localization`) publishing the same topic would drop in without changing the controller. Sensors only ever feed the estimator, never the controller.

10 Results

The route below was flown with `goto.py`:

```
0,0,1.5 → 3,3,2 → -2,1,1,90° → 2,-2,2,90° → 0,0,2,-90° → 0,0,2,180° → 0,0,1.5
```

A waypoint counts as reached within 0.15 m, below 0.2 m/s, and within 5° of yaw.

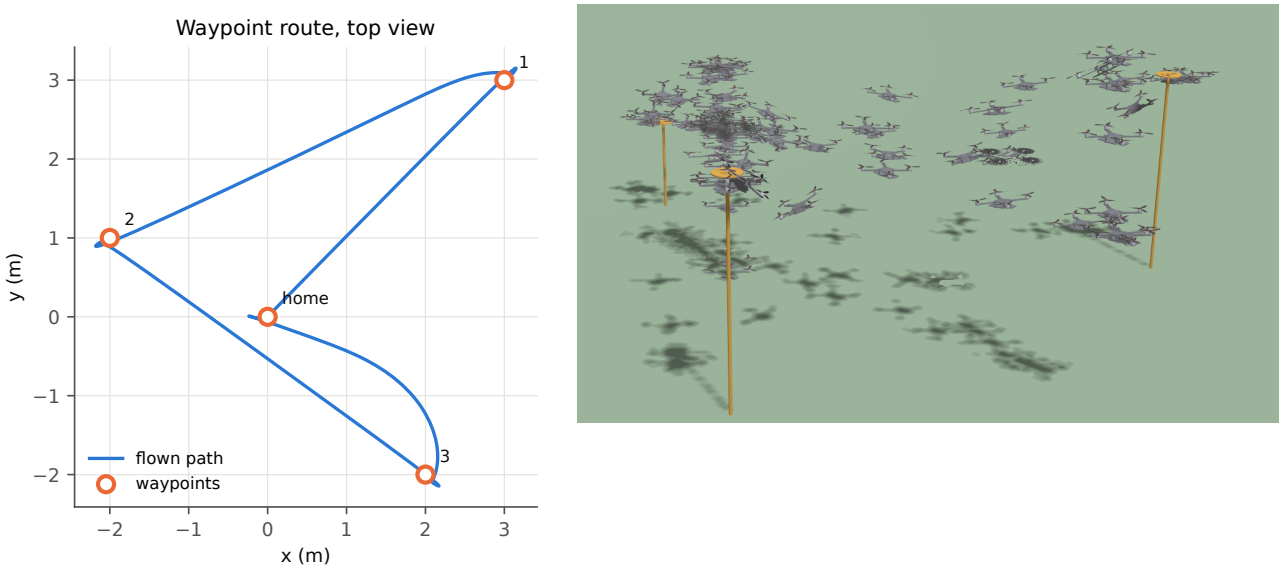


Figure 8. The route from above (left), and as seen by a fixed camera with every half second of the flight overlaid (right).

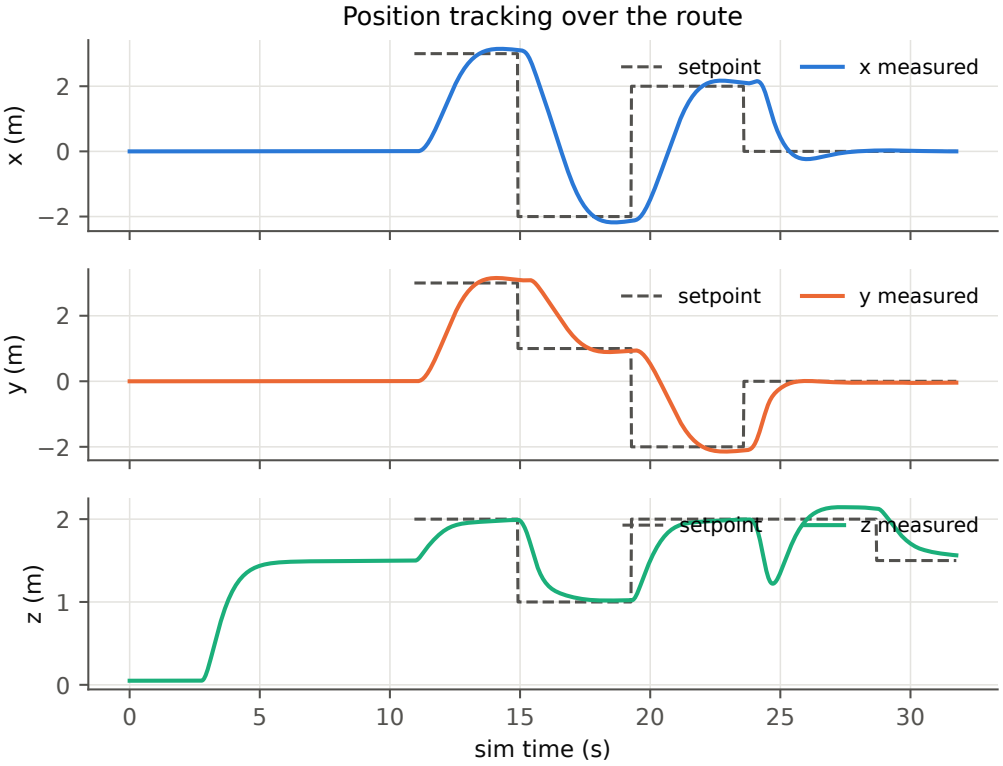


Figure 9. Position against its setpoint over the route.

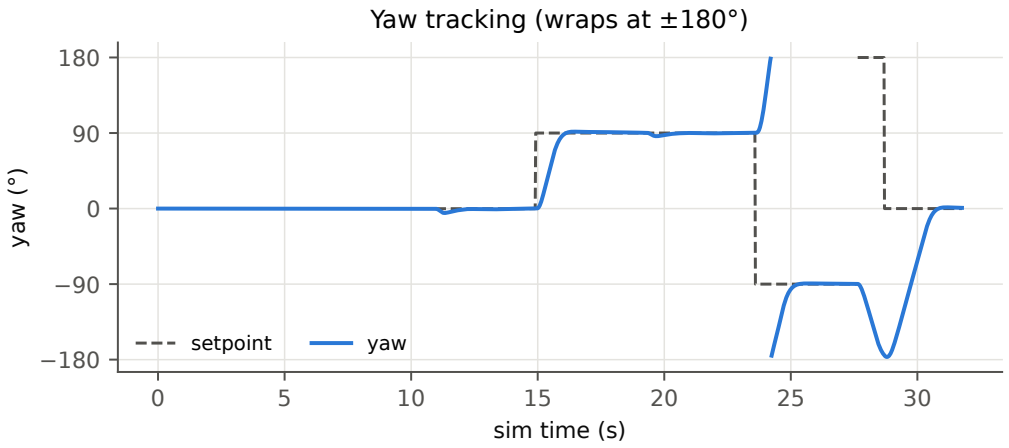


Figure 10. Yaw against its setpoint over the route.

Waypoint	Time to reach (s)	Overshoot (m)
(3, 3, 2)	4.0	0.21
(−2, 1, 1), yaw 90°	4.4	0.20
(2, −2, 2), yaw 90°	4.3	0.22
(0, 0, 2), yaw −90°	4.0	0.17
(0, 0, 2), yaw 180°	1.1	0.00
(0, 0, 1.5), yaw 0°	1.9	0.00

Waypoint performance (simulation time).

11 Compared with Gazebo's built-in controller

`controller:=gz` still selects Gazebo's `MulticopterVelocityControl`, a geometric controller of the Lee et al. type. Its loops are velocity → attitude → rate, all proportional-derivative.

	Gazebo's controller	This controller
Position loop	none: zero stick means zero velocity, so it slowly drifts	brakes, then holds a point
Integral terms	none: model errors leave a steady offset	velocity and rate integrators remove them
Thrust priority	fixed tilt cap	vertical first, thrust headroom kept
Yaw when saturated	no special handling	largest share that fits
Go-to-goal	not possible	built in (`/goal_pose`)
Code	closed inside a Gazebo plugin	`control.hpp`, unit tested

12 Tuning guide

Tune from the inside out: rate, then attitude, then velocity, then position. Keep each outer loop about 3–5× slower than the loop inside it.

Symptom	Change in `config/controller.yaml`
Fast wobble or buzz in hover	lower `rate_p`, or raise `rate_d` slightly
Slow, lazy attitude	raise `att_p` (keep it below `rate_p`/3)
Overshoots waypoints	lower `vel_max_xy` or `pos_p`, or raise `vel_p`
Sags or drifts in altitude	raise `vel_i` on z
Slow, weak yaw	raise `att_p`/`rate_p` on yaw (it is drag-limited)

Changes take effect after `colcon build` and a relaunch. The unit tests (`colcon test --packages-select my_drone`) check the mixer's signs and scale, the attitude error signs and the tilt limit.

References

1. PX4 Autopilot, *Controller Diagrams*. docs.px4.io/v1.16/en/flight_stack/controller_diagrams
2. D. Brescianini, M. Hehn, R. D'Andrea, *Nonlinear Quadrocopter Attitude Control*, ETH Zurich, 2013.
3. T. Lee, M. Leok, N. H. McClamroch, *Geometric Tracking Control of a Quadrotor UAV on SE(3)*, IEEE CDC, 2010.
4. D. Mellinger, V. Kumar, *Minimum Snap Trajectory Generation and Control for Quadrotors*, IEEE ICRA, 2011.